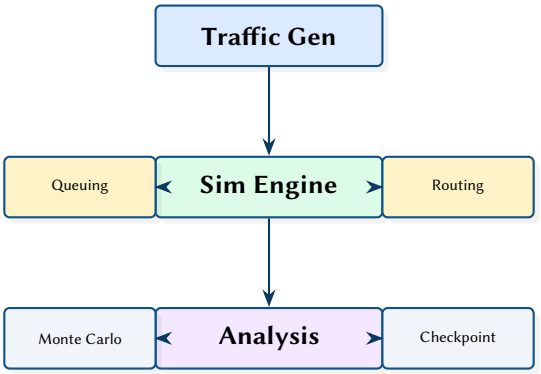
NetFlowSim: A Discrete Network Flow Simulator with Queuing Models and Monte Carlo Analysis

Traffic Generation, Routing, Failure Cascades, and Checkpoint/Restore

Simon Knight

March 2026 • Version 0.1.0



Abstract. NetFlowSim is a Rust-based discrete network flow simulator that models traffic flows across graph topologies with pluggable queuing models (M/M/1, M/D/1, M/M/c, M/G/1), configurable service time distributions (Exponential, Pareto, LogNormal, Weibull), and weighted multi-path routing. The simulator supports Monte Carlo analysis with parallel execution via Rayon, deterministic seeding for reproducibility, and binary checkpoint/restore for resumable long-running simulations. An analysis framework provides N-1 resilience testing, bottleneck detection, capacity planning, cascade failure modelling, correlation analysis, and regional traffic analytics.

Version	Date	Changes
0.1.0	March 2026	Initial architecture report

Contents

List of Figures

List of Tables

List of Design Decisions & Rules

1 Introduction and Motivation

Understanding network behaviour under load requires simulation that captures both the topology structure and the queuing dynamics at each node. NetFlowSim combines a graph-based network model with classical queuing theory to simulate traffic flows, measure per-link utilisation, and identify failure modes through systematic analysis.

2 Simulation Engine

The engine is built on a `petgraph::StableGraph` [1] with indexed simulation for performance. Nodes and links can be dynamically disabled/enabled to model failures, and a routing cache avoids recomputation when the topology is unchanged.

```rust
pub struct NetworkSimulator {
    graph: StableGraph<NodeState, LinkState>,
    routing_matrix: Option<RoutingMatrix>,
    flows: Vec<Flow>,
    rng: ChaCha8Rng,
    stats: SimulationStats,
}
```

2.1 Queuing Models

Four queuing models are available, selected per-node:

Table 1. Supported queuing models.

Model	Parameters	Delay Formula
M/M/1	λ, μ	$\frac{1}{\mu-\lambda}$
M/D/1	λ, μ	$\frac{2-\rho}{2\mu(1-\rho)}$
M/M/c [2]	λ, μ, c	Erlang-C based
M/G/1 [3]	λ, μ, C_s^2	Pollaczek–Khinchine

The M/G/1 model accepts arbitrary service time distributions through a trait-based design:

```rust
pub trait ServiceDistribution: Send + Sync {
    fn mean(&self) -> f64;
    fn cv_squared(&self) -> f64;  // Coefficient of variation squared
    fn sample(&self, rng: &mut impl Rng) -> f64;
    fn cdf(&self, x: f64) -> f64;
}
```

Implementations include Exponential, Deterministic, Pareto, LogNormal, and Weibull distributions.

> **Design Decision 1: Pluggable Service Distributions**
>
> By parameterising queuing models with a `ServiceDistribution` trait rather than hardcoding distribution types, NetFlowSim can model both the heavy-tailed traffic patterns typical of internet workloads (Pareto) and the deterministic processing times of hardware forwarding engines.

3 Traffic Generation

The traffic generator produces flows with configurable source/destination selection. A region-aware locality bias ensures that a configurable fraction of traffic stays within the same region, modelling realistic enterprise traffic patterns.

3.1 Routing

The routing module generates a full routing matrix using weighted shortest paths. Multi-path routing distributes flows across Equal-Cost Multi-Path (ECMP) paths with configurable load-balancing weights.

4 Analysis Framework

Six analysis modules operate on simulation results:

Table 2. Analysis modules.

Module	Description
N-1 Resilience	Tests every single-element failure; reports impact
Bottleneck	Identifies links/nodes with highest utilisation
Capacity Planning	Projects when links will reach capacity under growth
Cascade Failure	Models progressive failure under overload
Correlation	Finds correlated failure patterns
Regional	Per-region traffic flow analysis

4.1 Monte Carlo Simulation

The Monte Carlo module runs parallel iterations using Rayon [4] with deterministic per-iteration seeding:

```
pub fn run_monte_carlo(
    config: &MonteCarloConfig,
    topology: &Topology,
) -> MonteCarloResults {
    (0..config.iterations)
        .into_par_iter()
        .map(|i| {
            let seed = config.base_seed ^ i.wrapping_mul(0x9E3779B97F4A7C15);
            let mut sim = NetworkSimulator::new(topology.clone(), seed);
            sim.run_iteration(&config.scenario)
        })
        .collect()
}
```

: Deterministic Monte Carlo

Each iteration derives its Random Number Generator (RNG) seed deterministically from the base seed and iteration index. Results are identical regardless of Rayon's thread scheduling, enabling reproducible analysis.

5 Checkpoint and Restore

Long-running simulations can be checkpointed to disk using binary serialisation (bincode). The checkpoint captures the full simulator state: topology, flow state, RNG seeds, routing matrix, and accumulated statistics.

6 Scheduled Events

The event system supports time-scheduled link and node failures with configurable convergence modelling. Events can be defined as:

- **Deterministic** — Fixed time, fixed target
- **Stochastic** — Poisson-distributed failure times
- **Correlated** — Failures triggered by other failures (shared risk link groups)

Insight: *Checkpoint files include a schema version field, enabling forward-compatible deserialisation as the simulator evolves. Unknown fields in newer schemas are silently ignored when loading older checkpoints.*

7 Design Summary

Table 3. Key design decisions.

Decision	Rationale
StableGraph	Node/link disable without index invalidation
Trait-based distributions	Pluggable service times for diverse workloads
ChaCha8Rng	Cross-platform deterministic simulation
Binary checkpoints	Fast serialisation for multi-hour simulations
Rayon parallelism	Near-linear Monte Carlo scaling across cores

References

[1] bluss, *Petgraph: Graph data structure library for Rust*, 2024. [Online]. Available: https://crates.io/crates/petgraph

[2] A. K. Erlang, *Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges*. 1917.

[3] F. Pollaczek, "Über eine aufgabe der wahrscheinlichkeitstheorie," *Mathematische Zeitschrift*, 1930.

[4] Rayon contributors, *Rayon: Data parallelism library for Rust*, 2024. [Online]. Available: https://crates.io/crates/rayon